

I've created your PDF with all 5 cybersecurity practical explanations.

 [Download your PDF](#)

Now I'll give you the **FULL DETAILED EXPLANATION** (very beginner-friendly, like you prefer) 

1. S-DES (Simplified XOR Encryption)

◆ Code Concept

This is not real DES (which is complex), but a **simplified version using XOR** to demonstrate encryption.

◆ Line-by-line Understanding

```
def encrypt(plain, key):
```

- Function to encrypt message
- plain = original text
- key = secret key

```
cipher = ""
```

- Empty string to store encrypted text

```
for i in range(len(plain)):
```

- Loop runs for each character in text

```
ord(plain[i])
```

- Converts character → ASCII number
Example: 'A' → 65

```
ord(key[i % len(key)])
```

- Key repeats if shorter than text
- % ensures looping of key

```
^
```

- XOR operation
- Core encryption logic

```
chr(...)
```

- Converts ASCII back to character
-

◆ Important Concept

👉 XOR property:

$$A \oplus B \oplus B = A$$

That's why **same function works for decryption**

🔒 2. AES (Advanced Encryption Standard)

◆ Concept

AES is:

- Symmetric encryption
 - Uses same key for encryption & decryption
 - Much stronger than DES
-

◆ Code Explanation

```
from Crypto.Cipher import AES
```

- Imports AES algorithm

```
key = get_random_bytes(16)
```

- Generates 16-byte key (128-bit)

```
cipher = AES.new(key, AES.MODE_EAX)
```

- Creates AES object
- EAX mode = secure mode

```
ciphertext, tag = cipher.encrypt_and_digest(data)
```

- Encrypts data
 - Generates authentication tag
-

◆ Decryption

```
cipher = AES.new(key, AES.MODE_EAX, nonce=cipher.nonce)
```

- Same key + nonce required

```
plaintext = cipher.decrypt(ciphertext)
```

- Converts back to original text
-

🔒 3. Diffie-Hellman Key Exchange

◆ Concept

👉 Used to **share secret key over public network**

◆ Code Explanation

$p = 23$

$g = 5$

- Public values (everyone knows)

$a = \text{random.randint}(1, 10)$

- Alice private key (secret)

$A = (g^{**} a) \% p$

- Alice public value

$b = \text{random.randint}(1, 10)$

$B = (g^{**} b) \% p$

- Bob values
-

◆ Secret Key

$\text{key_A} = (B^{**} a) \% p$

$\text{key_B} = (A^{**} b) \% p$

👉 Both get SAME key

🔥 Why secure?

- Based on **Discrete Logarithm Problem**
 - Hard to reverse
-

🔒 4. RSA Algorithm

◆ Concept

- Asymmetric encryption
- Uses **2 keys**
 - Public key (share)
 - Private key (secret)

◆ Step-by-step

$p = 3$

$q = 11$

- Prime numbers

$n = p * q$

- Used in both keys

$\phi = (p - 1) * (q - 1)$

- Important for key generation
-

◆ Public Key

$e = 3$

Condition:

$\gcd(e, \phi) = 1$

◆ Private Key

$d = \text{mod_inverse}(e, \phi)$

◆ Encryption

$c = (\text{msg} ** e) \% n$

◆ Decryption

$m = (c ** d) \% n$

5. ECC (Elliptic Curve Cryptography)

◆ Concept

- Uses **elliptic curve math**
 - More secure than RSA with smaller keys
-

◆ Code Explanation

a = 2

b = 3

- Curve equation:

$$y^2 = x^3 + ax + b$$

G = (3, 6)

- Generator point

d = random.randint(1, 10)

- Private key

Q = (d * G[0] % p, d * G[1] % p)

- Public key

Key Idea

 Instead of multiplication → uses **point multiplication**

FINAL REVISION (VERY IMPORTANT)

Algorithm	Type	Key Idea
S-DES	Symmetric	XOR encryption
AES	Symmetric	Fast & secure
Diffie-Hellman Key Exchange		Shared secret
RSA	Asymmetric	Public + Private key
ECC	Asymmetric	Smaller keys, high security

If you want next:

I can:

- Add **diagrams for each (exam-friendly)**
- Give **perfect viva answers (topper level)**
- Explain **line-by-line code even deeper (like what is ord, chr, mod, etc.)**

Just tell 